Sprintfour Hackathon

Participant Handout

About this challenge

Conseal is a desktop application built by Sprintfour that anonymizes documents by automatically redacting or labeling personally identifying information (PII), so the documents can be safely shared with AI tools without leaking private data. For this hackathon, you will build a full-stack application that tackles one real problem a tool like Conseal has to solve well.

> **A note on local vs cloud:** the real Conseal application runs 100% locally, with no data ever leaving the user's machine. For the purpose of this hackathon, however, using cloud solutions and APIs (including cloud LLMs) is fully allowed, so you can focus on the experience rather than on local infrastructure.

Pick ONE of the three problem statements below. Each describes a real person with a real problem and a goal to reach. We are intentionally not handing you a feature checklist. Part of the challenge is deciding what to build. The strongest submissions are the ones that understand the user deeply and handle the cases that are not spelled out.

Format and timeline

- **Solo.** You work individually.
- **Build day.** 8 hours on event day to produce a working prototype.
- **Extended submission.** You may keep refining and submit any time before end of week. The competition concludes on the day, but the extended window gives you a chance to go deeper. Top submissions are invited to interview with the company.
- **Use AI freely.** You are expected to use AI tools for development. Because everyone has them, a working prototype is the baseline. What sets your submission apart is judgment: which hard cases you noticed, and how thoughtfully you handled them.

What to build

A full-stack application. That means a real frontend (the experience the user interacts with) and a real backend (the logic and services behind it), wired together and runnable. The heart of every problem here is the user experience and the thinking behind it, so invest in how it feels to use, not only in whether it runs.

PII detection: two easy options

You do NOT need to build a PII detector from scratch. Detection is a means to an end here, not the point. Choose whichever path lets you focus on the actual problem:

- **Option A. Cloud LLM.** Use any cloud LLM with your own API key to detect PII in the document text. A simple prompt that returns the sensitive spans and their types is enough. This gives you realistic, messy detection to design around.
- **Option B. Mock backend.** Use a mock backend that returns a fixed list of detected PII spans (text, type, and a confidence score) for a sample document. This removes all setup friction so you can spend your time on the experience.

Either option is fully accepted and neither is scored higher than the other. Pick the one that gets you to the interesting work fastest.

Problem 1: Trust and explainability

> *Marcus has a sensitive document he wants to paste into an AI tool, but he is nervous. He has been burned before: he has heard stories of "redacted" documents where the information was still there underneath. On top of that, he does not trust tools he cannot interrogate. Every time something is hidden, he wants to know why it was hidden, and when something is left visible, he wants to know why it was kept. He will not adopt a tool he has to take on faith.*

You are given a document and the redactions our tool applied (with the type and confidence behind each one). Build the experience that takes Marcus from anxious to confident, and that answers his constant question: why this, and why not that?

We are not looking for a specific feature set. We want to see how you think about earning the trust of a worried, skeptical person.

Problem 2: Working at volume

> *Maya is a paralegal. She has 200 case files to anonymize before end of day, and right now she does them one document at a time. She is fast, she is under pressure, and she is exactly the kind of user who will abandon a tool the moment it slows her down.*

You are given a set of documents and the redactions our tool suggests for each. Build the experience Maya actually wants.

We are not prescribing what that looks like. Show us you understand a high-volume user working under real time pressure.

Problem 3: Fixing the tool's mistakes

> *Sam is reviewing a tool's suggested redactions. The tool has hidden a few things that are not sensitive at all, and, worse, it has left a phone number and a name untouched. Sam is moving fast and trusts the tool a little too much. The mistakes that slip through are the ones he does not stop to look at.*

You are given a document and our tool's suggested redactions, which contain both false positives (harmless text that was hidden) and missed PII (sensitive text left visible). Build the correction experience for someone working quickly and imperfectly.

We are not telling you what it should look like. Show us you understand how people actually behave when correcting a machine.

What to submit

- **The application.** Your full-stack app, runnable, with clear instructions to start it or the hosted site URL if available.
- **A short writeup (about half a page).** What you built, and just as important, what you chose NOT to build and why. With AI handling much of the typing, your prioritization and reasoning are the clearest signal of how you think.
- **A short video recording.** A brief walkthrough (video or live) showing the experience in action.
- **Your resume.** Add this to your GitHub repo for submission.

How submissions are judged

Working code is the floor, not the differentiator. We are looking at:

- **Software engineering fundamentals.** You are computer science students, and we want to see that you have strong fundamentals: clean, well-structured code, sensible architecture and separation of concerns, clear naming, error handling, and a project a teammate could pick up and understand. AI can help you write code, but sound engineering judgment is what we are evaluating.
- **Discovery.** Did you notice the hard cases that were not spelled out in the prompt?
- **Judgment.** Did you spend your effort on the part of the problem that actually matters?
- **Real-user empathy.** Did you design for the actual person under their actual constraints, or for a clean demo?
- **Tradeoff awareness.** Did you recognize the tensions in the problem and make a deliberate call?
- **Reasoning.** Can you explain your choices, including what you left out?